

Computer Organization and Architecture



Introduction

- Overview of computer organization and architecture
- Importance of understanding computer history and structure

What is a Computer?

- A computer is an electronic machine that processes information.
- It can perform tasks like calculations, storing data, and running programs.
- Examples: Smartphones, Laptops, Gaming Consoles.

History of Computers

Early Computers:

- The first computers were large machines that used vacuum tubes.
- They could only do simple calculations.

Evolution Over Time:

- Transistors replaced vacuum tubes, making computers smaller and faster.
- Microprocessors allowed personal computers to be created.
- Today, computers use AI and quantum computing to solve complex problems.

Generations of Computers

First Generation (1940s - 1950s)

- Used vacuum tubes.
- Very large and expensive.
- Example: ENIAC.

Second Generation (1950s - 1960s)

- Used transistors (smaller and more reliable).
- Faster than vacuum tube computers.
- Example: IBM 1401.

Generations of Computers

Third Generation (1960s - 1970s)

- Used integrated circuits (ICs).
- Smaller, faster, and cheaper.
- Example: IBM System/360.

Fourth Generation (1970s - Present)

- Uses microprocessors (a whole CPU on a single chip).
- Personal computers became popular.
- Example: Intel 4004, Apple Macintosh.

Generations of Computers

Fifth Generation (Present & Future)

- Uses AI and quantum computing.
- Focuses on automation and high-speed processing.
- Example: Supercomputers, AI-based processors.

How Does a Computer Work?

Basic Parts of a Computer:

- Input Devices – Keyboard, Mouse, Touchscreen.
- Processing Unit – CPU (Brain of the Computer).
- Storage – RAM (Temporary Memory) and Hard Drive (Permanent Memory).
- Output Devices – Monitor, Printer, Speakers.

Computer Operations:

- Input → Processing → Storage → Output
- Example: Typing a document → Computer processes text → Saves in memory → Displays on the screen.

Computer Architecture vs. Organization

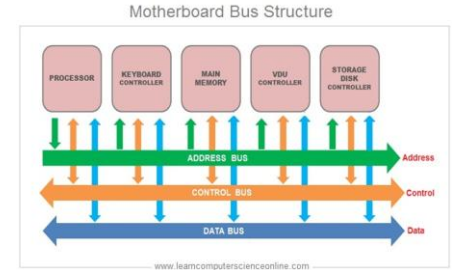
Feature	Computer Architecture	Computer Organisation
Definition	The design and function of a computer system.	The way a computer's hardware is built.
Focus	What a computer can do.	How a computer does it.
Example	Types of instructions, memory addressing.	CPU design, buses, connections.

What is a Bus in a Computer?

- A bus is like a road that carries data between different parts of a computer.
- It connects the CPU, memory, and other devices.
- Example: When you press a key, data travels through a bus to the CPU.

Types of Buses:

- Address Bus – Sends memory addresses.
- Data Bus – Carries actual data.
- Control Bus – Sends signals for controlling devices.



Common Bus Systems

Industry Standard Architecture (ISA)

- One of the oldest computer bus systems.
- Used in early IBM PCs.
- Slow compared to modern buses.
- Example: Used for connecting floppy disk controllers and early sound cards.

Common Bus Systems

Peripheral Component Interconnect (PCI)

- A faster and more flexible bus standard.
- Used in desktop computers for connecting hardware like graphics cards and sound cards.
- Example: Modern PCI Express (PCIe) is used for high-speed graphics and SSDs.

Common Bus Systems

Universal Serial Bus (USB)

- A common bus for connecting external devices.
- Supports keyboards, mice, flash drives, and external hard drives.
- Allows plug-and-play functionality.
- Example: USB 3.0 provides fast data transfer speeds.

Common Bus Systems

Advanced Microcontroller Bus Architecture (AMBA)

- Used mainly in mobile devices and embedded systems.
- Designed for low power consumption and efficiency.
- Example: Used in smartphones and tablets to connect components like processors and cameras.

How Buses Work

How Do Buses Help Computers?

- Transfer Information Quickly – Like a highway moving cars.
- Allow Communication Between Parts – CPU, Memory, and Hard Drive all talk using buses.
- Control How Data Moves – Prevents two devices from sending data at the same time.

Bus Arbitration (Who Goes First?)

Definition: The process of deciding which device gets to use the shared bus when multiple devices request access at the same time.

- Daisy Chaining: Devices take turns based on priority.
- Polling: The CPU asks each device if it needs to send data.
- Independent Request: Each device sends a request, and the CPU decides who goes first.

How Buses Work

Bus Arbitration (Who Goes First?)

Daisy Chaining:

- Devices take turns based on priority.
- The highest-priority device gets access first.
- Example: Some older bus systems use this method for selecting devices.

Polling:

- The CPU asks each device in sequence if it needs to send data.
- Simple but slow because it cycles through all devices.
- Example: USB devices may use polling for communication with the CPU.

How Buses Work

Bus Arbitration (Who Goes First?)

Independent Request:

- Each device sends a request to the bus controller.
- The bus controller decides who goes first based on priority and fairness.
- Example: Modern PCIe buses use this method for fast and efficient communication.

Time Division Multiplexing (TDM):

- Each device gets a fixed time slot to send data.
- Reduces waiting time but may waste time slots if a device has nothing to send.
- Example: Some real-time systems use TDM to ensure predictable data transfer.

How Buses Work

Bus Arbitration (Who Goes First?)

Priority Arbitration:

- Devices have assigned priority levels.
- High-priority devices can interrupt lower-priority ones.
- Example: Used in high-speed buses like PCI Express to manage data traffic efficiently.

How Buses Work

Bus Arbitration (Who Goes First?)

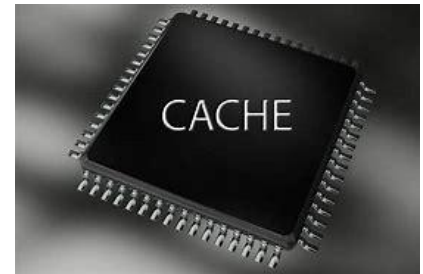
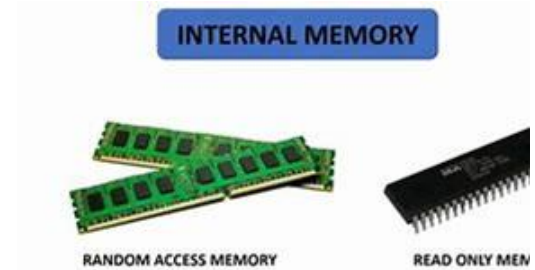
Priority Arbitration:

- Devices have assigned priority levels.
- High-priority devices can interrupt lower-priority ones.
- Example: Used in high-speed buses like PCI Express to manage data traffic efficiently.

Internal Memory

Types of Memory:

- RAM (Random Access Memory): Temporary storage for running programs.
- ROM (Read-Only Memory): Permanent memory that holds startup instructions.
- Flash Memory: Used in USB drives and SSDs.
- Virtual Memory: A part of the hard drive used as extra RAM.
- Cache Memory: High-speed memory close to the CPU that stores frequently used data for quick access.



Internal Memory

Memory Building Blocks:

- Bits & Bytes: Smallest units of data storage.
- Memory Cells: Store 1s and 0s (binary data).
- Memory Modules: Groups of memory chips (e.g., Double Data Rate (DDR) RAM).
- Memory Banks: Organized storage areas for faster access.
- Memory Controllers: Manage data flow between memory and CPU.
- Memory Addressing: Determines how data locations are identified and accessed by the CPU.

Registers

Definition: Small, fast memory locations inside the CPU.

- However, it is a small device that can hold only one word of data.
- The one-word data is a standard data unit.
In most the it is a 8-bit word i.e., byte

Purpose: Store instructions and data for quick processing.

Types of Registers:

- **Accumulator(ACC):** Holds arithmetic calculations.
- **Instruction Register(IR):** Stores current instruction.
- **Stack Pointer(SP):** Keeps track of function calls.
- **Program Counter(PC):** Tracks the next instruction to be executed.

Cache Memory

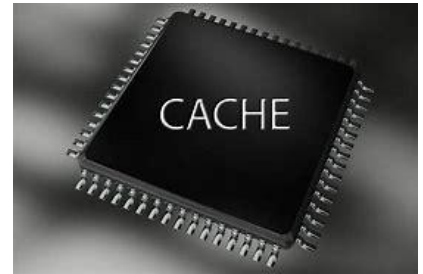
Definition: A small, high-speed memory close to the CPU that stores frequently used data and instructions.

Levels:

- L1 (Level 1): Smallest and fastest, built into the CPU.
- L2 (Level 2): Larger, slightly slower, on or near the CPU.
- L3 (Level 3): Largest, slowest, shared among multiple cores.

Cache Replacement Policies:

- **Least Recently Used (LRU):** Removes the least-used data.
- **First In, First Out (FIFO):** Removes oldest stored data.
- **Random Replacement:** Replaces data randomly when full.



Cache Memory

Definition: A small, high-speed memory close to the CPU that stores frequently used data and instructions.

Example: If the CPU needs the same data multiple times (such as a value in a mathematical calculation), it will fetch that data from the cache rather than searching through slower RAM or storage.

Levels:

- L1 (Level 1): Smallest and fastest, built into the CPU.
- L2 (Level 2): Larger, slightly slower, on or near the CPU.
- L3 (Level 3): Largest, slowest, shared among multiple cores.

Cache Replacement Policies:

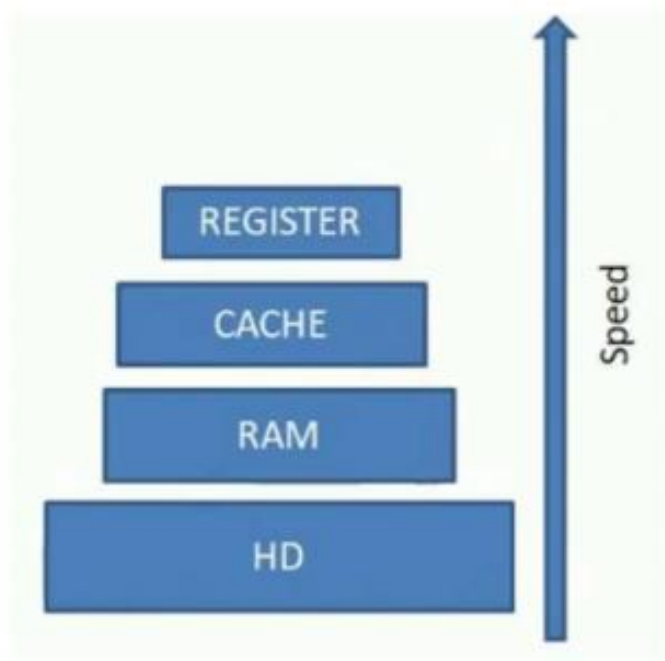
- **Least Recently Used (LRU):** Removes the least-used data.
- **First In, First Out (FIFO):** Removes oldest stored data.
- **Random Replacement:** Replaces data randomly when full.

Registers vs Cache Memory

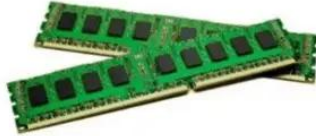
Difference Between Cache and Register

- To explain in simple words, Registers store the data before processing, and Cache Memory stores the data after progressing.

Cache Memory	Registers
Cache Stores Processed data.	Registers store Unprocessed data.
Cache Memory could be located inside the CPU or on the motherboard.	Registers are a part of the CPU.
CPU uses Cache to detect previously stored data.	CPU uses registers to process new data.
Cache Memory Stores recently used data.	Registers store the data that the CPU is currently processing.



INTERNAL MEMORY



RANDOM ACCESS MEMORY



READ ONLY MEMORY

Computer Memory And Its Types

EXTERNAL MEMORY



Memory Card Reader



USB Flash
Memory



Media
Devices



External Optical Drives



ZIP Drive

External Memory

- **External memory** refers to devices that store data outside the main computer system.
- These devices are used for **long-term data storage** and can be easily added or removed from the system.
- **Types of external memory** include hard disks, optical drives, and solid-state drives.

Storage Device Characteristics

Key Characteristics to Consider:

- **Capacity:** How much data the device can hold.
- **Speed:** How quickly data can be read or written.
- **Durability:** How long the device lasts.
- **Portability:** How easily the device can be moved or connected.
- **Cost:** How affordable the device is.

Storage Device Characteristics

Selecting a Storage Device:

- Depending on the need (speed, capacity, portability), you choose the right storage device.
- Example: For large data storage, **external hard disks** are selected, while for fast access and durability, **SSDs** are preferred.

Hard Disk Storage

- **Definition:** Hard Disk Drives (HDDs) are mechanical storage devices that use spinning disks to read/write data.
- **How it Works:** A moving read/write head accesses data on the spinning platters.
- **Advantages:** Larger storage capacities at lower prices.
- **Disadvantages:** Slower speeds compared to SSDs and more prone to physical damage.
- **Example:** HDDs are commonly used for desktop storage or backup drives.



How to Use:

- Simply connect the HDD to the computer using a USB port.

Magnetic Storage

- **Definition:** Magnetic storage uses magnetic fields to write and read data.
- **How it Works:** Data is stored on a magnetized medium (e.g., tape or hard disk platter).
- **Types of Magnetic Storage:**
 - a. **Floppy Disks:** Outdated, but historically used for transferring data.
 - b. **Magnetic Tapes:** Used for backup and archiving due to their large storage capacity.
 - c. **Hard Disk Drives (HDDs):** As described above.

Advantages of Magnetic Storage:

- Cost-effective for large storage.
- Durable with proper handling.

Optical Mass Storage Devices

Definition: Optical storage uses light (lasers) to read and write data on discs.

Types of Optical Storage Devices:

- a. **CDs (Compact Discs):** Typically store music or small files.
- b. **DVDs (Digital Versatile Discs):** Can store larger files like videos or software.
- c. **Blu-ray Discs:** Higher capacity, used for high-definition video storage.

How it Works: A laser reads or writes data to/from the disc surface.

Advantages of Optical Storage:

- Relatively cheap and easy to mass-produce.
- Good for archiving data.



Solid State Disk (SSD)

Definition: SSDs are storage devices that use **flash memory** to store data.

How it Works: Data is stored in **non-volatile flash memory chips** instead of spinning disks.

Advantages: Faster than HDDs, more durable, and quieter.

Disadvantages: Higher cost per GB compared to HDDs.

Example: Used in modern laptops and high-performance desktops.



How to Use:

- Simply connect the SSD to the computer using a USB or internal connection.

Non-Volatile Memory Express:(NVMe)

- **Definition:** NVMe is a **protocol** designed to enable faster communication between the CPU and storage devices (like SSDs) through the **PCIe (Peripheral Component Interconnect Express)** interface.
- **How it Works:** NVMe uses the high-speed PCIe bus to access storage devices directly, which significantly speeds up data transfer compared to older storage protocols like **SATA**.
- **Example:** NVMe SSDs are used for high-speed storage in gaming PCs, workstations, and enterprise systems.



SSD vs NVMe

Key Differences:

- **SSD** refers to the **type of storage** device, while **NVMe** refers to the **communication protocol** that makes SSDs faster.
- **All NVMe drives are SSDs**, but not all SSDs are NVMe. Some SSDs use the older **SATA** interface (which is slower than PCIe).

External Memory Configuration Technologies

Types of Configurations:

- **RAID (Redundant Array of Independent Disks):** Combines multiple drives for improved performance or data redundancy.
- **NAS (Network Attached Storage):** A server that provides external storage over a network.
- **Cloud Storage:** Remote servers store your data, accessible via the internet.

RAID (Redundant Array of Independent Disks)

What is RAID?

- Combines multiple hard drives or SSDs into a single storage unit for increased speed, redundancy, or both.

How Does RAID Work?

- Data is distributed across drives in different ways.
- **Common RAID Levels:**
 - **RAID 0 (Striping):** Increases speed, but no redundancy.
 - **RAID 1 (Mirroring):** Provides redundancy by duplicating data.
 - **RAID 5 (Striping with Parity):** Balances speed and redundancy.
 - **RAID 10 (1+0):** Combines RAID 1 and RAID 0 for high performance and redundancy.

Example Use Case:

- RAID 5 is often used in servers for **speed** and **data protection**.

NAS (Network Attached Storage)

What is NAS?

- A device that connects to a network and provides shared storage for multiple users.

How Does NAS Work?

- Acts as a **centralized storage device** accessible by multiple devices over a network (LAN).
- Often includes **RAID** support for increased data protection and speed.

Benefits of NAS:

- **Centralized Storage:** All files stored in one location, accessible from multiple devices.
- **Data Redundancy:** Many NAS devices use RAID for data protection.

Example Use Case:

- Used in **small businesses** for file sharing and in homes for **media storage**.

Cloud Storage

What is Cloud Storage?

- A remote storage service accessible over the internet, hosted by a service provider.

How Does Cloud Storage Work?

- Data is stored on remote servers and accessed via the internet from any connected device.

Popular Cloud Services:

- **Google Drive, Dropbox, OneDrive.**

Cloud Storage

Benefits of Cloud Storage:

- **Easy Access:** Available anytime and anywhere with internet.
- **Scalable:** Pay only for the amount of storage you need.
- **Data Backup:** Provides off-site backup to prevent data loss from local failures.

Example Use Case:

- **Individuals** use it for file storage, and **businesses** use it for remote work and data backup.

Hardware Operating System Support

I/O Modules

- Connect external devices to the CPU and memory.
- Control and manage the transfer of data.
- Example: Keyboard controller, disk controller.

Interrupts

- An **interrupt** is a signal that temporarily stops a running program so the CPU can attend to something urgent.
- Example: You click a mouse — the CPU pauses what it's doing to process the click.
- Helps manage multiple tasks efficiently.

Interrupts

- An **interrupt** is a signal that temporarily stops a running program so the CPU can attend to something urgent.
- Types of Interrupts:
 - Hardware Interrupts: From external devices (keyboard, mouse, etc.).
 - Software Interrupts: Generated by programs or the operating system.
 - Timer Interrupts: Triggered at regular intervals to manage tasks.
- Example:
 - You click a mouse — the CPU pauses what it's doing to process the click.
 - A game requests to save progress → software interrupt tells the OS to access the disk.
 - While typing in a document, your computer automatically checks for software updates in the background.
- Helps manage multiple tasks efficiently.

Interrupts

- An **interrupt** is a signal that temporarily stops a running program so the CPU can attend to something urgent.
- Types of Interrupts:
 - Hardware Interrupts: From external devices (keyboard, mouse, etc.).
 - Software Interrupts: Generated by programs or the operating system.
 - Timer Interrupts: Triggered at regular intervals to manage tasks.
- Example:
 - You click a mouse — the CPU pauses what it's doing to process the click.
 - A game requests to save progress → software interrupt tells the OS to access the disk.
 - While typing in a document, your computer automatically checks for software updates in the background.
- Helps manage multiple tasks efficiently.

Direct Memory Access (DMA)

- Allows devices to send/receive data to/from memory without CPU involvement.
- Improves speed by freeing the CPU.
- Example: Transferring data from a hard drive to RAM.

Direct Memory Access (DMA)

- Definition: A method that allows devices to transfer data directly to/from memory without involving the CPU.

How DMA Works:

- A device like a hard drive or network card sends a DMA request to the memory controller.
- The controller grants access and data is transferred directly between the device and memory.
- The CPU is only notified once the transfer is complete (via an interrupt).

Direct Memory Access (DMA)

Why DMA is Important:

- **Speeds Up Performance:** Frees up CPU to handle other tasks.
- **Efficient for Large Transfers:** Ideal for moving blocks of data, such as loading a file into RAM.

Direct Memory Access (DMA)

Example:

- Playing a video from a USB drive: the video is loaded into RAM using DMA while the CPU handles playback controls.
- When listening to music, audio data is streamed from memory to the sound card using DMA. CPU is free to handle other user tasks like browsing.
- When downloading a large file, the network card uses DMA to move data directly to system memory. Reduces CPU workload during file downloads.
- High-end games or video editing software use DMA to load textures from memory to the GPU quickly. Boosts performance by avoiding CPU bottlenecks.

I/O Channels and Processors

- Dedicated processors for input/output tasks.
- Relieve main CPU workload.
- Common in servers and mainframes.

External Devices

- Devices outside the computer performing I/O functions.
- Examples: Mouse, printer, external storage.
- Connect via USB, HDMI, Ethernet, etc.

Computer Arithmetic - Number Systems

- Number systems used by computers:
 - Decimal (Base 10): Human system.
 - Binary (Base 2): Used by computers.
 - Octal (Base 8): Compact form of binary.
 - Hexadecimal (Base 16): Simplifies binary representation.

Conversion Between Number Systems

- Conversions include:
 - Binary to Decimal and vice versa.
 - Octal/Hex to Binary.
- Essential for understanding data processing.

Character Codes

- Represent text using standardized codes:
 - ASCII: For English characters.
 - EBCDIC: IBM mainframe systems.
 - UNICODE: Supports global languages and symbols.

Design Instruction Sets

What is an Instruction Set?

- An Instruction Set is a collection of all the instructions a CPU can execute.
- It defines how software communicates with the hardware.
- Acts as the interface between hardware and software.
- Each instruction in the set is a binary-encoded command to perform a specific task.

Elements of Machine Instructions

- A machine instruction typically includes:
 - **Opcode**: Specifies the operation.
 - **Operand(s)**: Data or addresses involved in the operation.
 - **Addressing Mode**: Indicates how operands are accessed.
- Verified by analyzing bit structure and execution format.

Instruction Representation and Types

- **Instruction Format:**
 - Binary or hexadecimal encoding of operations.
 - Length varies (fixed or variable length instructions).
- **Instruction Types:**
 - **Data Transfer** (e.g., MOV, LOAD, STORE)
 - **Arithmetic/Logical** (e.g., ADD, SUB, AND)
 - **Control** (e.g., JUMP, CALL, RETURN)
 - **Input/Output** (e.g., IN, OUT)
- **Number of Addresses:**
 - **Three-address:** Two source operands, one destination.
 - **Two-address:** One operand is both source and destination.
 - **One-address:** Uses accumulator-based operations.
 - **Zero-address:** Stack-based operations.

Instruction Set Design, Operations, and Operands

- Instruction set design involves selecting:
 - A minimal yet complete set of instructions.
 - Consistency in opcode formats and operand access.
- Coordination:
 - Instructions must support efficient execution of operations.
 - Operands may be immediate values, memory addresses, or register references.
 - Operations should align with the processor's capabilities (e.g., pipelining, parallelism).
- Well-designed instruction sets improve performance, simplify compilers, and reduce hardware complexity.

Analyze the Difference Between CPU, RISC and CISC Architectures

Overview of CPU Architectures

- The CPU (Central Processing Unit) is the brain of the computer, responsible for executing instructions.
- RISC (Reduced Instruction Set Computing) and CISC (Complex Instruction Set Computing) are two major types of CPU architecture.

CISC Architecture Features

- Emphasis on hardware complexity.
- Multi-clock cycle complex instructions.
- Memory-to-memory operations (e.g., instruction can fetch and execute in one step).
- Smaller code sizes.
- High cycles per second.
- Greater number of transistors used for complex control logic.

Analyze the Difference Between CPU, RISC and CISC Architectures

RISC Architecture Features

- Emphasis on software simplicity.
- Single-clock cycle instructions.
- Register-to-register operations (load/store separated).
- Larger code sizes.
- Lower cycles per second per instruction.
- More transistors allocated for memory/register access and pipelining.

Analyze the Difference Between CPU, RISC and CISC Architectures

Example of Program Execution

- **CISC:** One instruction (e.g., `ADD A, B`) may fetch operands from memory, add, and store result.
- **RISC:** Requires multiple instructions (e.g., `LOAD R1, A; LOAD R2, B; ADD R3, R1, R2; STORE C, R3`).

